Competitive Programming

Section 50 Editorial: Advanced Algorithms Mixed Contest

Competitive Programming Course

2026-10-24

Table of contents

1 Contest coverage and reading order 3

2 A. DAG Grundy 4

3 B. General CRT 5

4 C. Fast Fibonacci 6

5 D. Fibonacci Range Sum 7

6 E. Pattern Prefix Matches 9

7 F. Suffix LCP Queries 11

8 G. Domino Placement 13

9 H. Weighted Tree Distances 16

10 I. Balanced Subset Difference 18

11 J. Monge Partition Table 20

12 K. Online Line DP 22

13 L. Profitable Disjoint Routes 25

14 M. Dial Alignment 28

15 N. Archive Reassembly 31

16 O. Relay Throughput 36

17 P. Last Alarm 43

18 Q. Exact Beacon Codes 47

19 R. Warehouse Blocks 51

20 S. Parabolic Signals 56

21 Testing discussion 61

22 What to carry forward 62

1 Contest coverage and reading order

The checker runs all nineteen local packages, A–S. A–D revisit earlier advanced topics; the following pairs provide two main problems for every technical section between mixed contests 42 and 50.

Source section	Topic	First problem	Second problem
43	String algorithms I	E. Pattern Prefix Matches	M. Dial Alignment
44	String algorithms II	F. Suffix LCP Queries	N. Archive Reassembly
45	Flows and matchings	G. Domino Placement	O. Relay Throughput
46	Rerooting and advanced tree DP	H. Weighted Tree Distances	P. Last Alarm
47	Meet-in-the-middle	I. Balanced Subset Difference	Q. Exact Beacon Codes
48	DP optimization	J. Monge Partition Table	R. Warehouse Blocks
49	Convex hull trick and Li Chao	K. Online Line DP	S. Parabolic Signals

L is an optional min-cost-flow extension, not a substitute for either of the two section-45 applications. Its extra algorithm is explained in its chapter. Read each chapter only after attempting its statement independently.

2 A. DAG Grundy

2.1 Restatement

For each start vertex in a DAG, print whether the first player wins.

2.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. □

2.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

2.4 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m,q;if(!(
```

2.5 Full Python solution

```python
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m,q=data[0],data[1],data[2]; idx=3; g=[[] for _ in range(n)]
    for _ in range(m): g[data[idx]-1].append(data[idx+1]-1); idx+=2
    gr=[0]*n
    for u in range(n-1,-1,-1):
        seen={gr[v] for v in g[u]}; x=0
        while x in seen: x+=1
        gr[u]=x
    out=["WIN" if gr[data[idx+i]-1] else "LOSE" for i in range(q)]
    print("\n".join(out))
if __name__=="__main__": main()
```

3 B. General CRT

3.1 Restatement

For each pair of congruences, print the smallest nonnegative solution and lcm modulus, or `NO`.

3.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. □

3.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

3.4 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std; long long eg(long long a,long long b,long long&x,long long&y){if(!b){x=
```

3.5 Full Python solution

```python
import sys,math
def eg(a,b):
    if b==0: return (abs(a),1 if a>=0 else -1,0)
    g,x,y=eg(b,a%b); return g,y,x-(a//b)*y
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    t=data[0]; idx=1; out=[]
    for _ in range(t):
        a,m,b,n=data[idx],data[idx+1],data[idx+2],data[idx+3]; idx+=4; g,x,y=eg(m,n)
        if (b-a)%g: out.append("NO")
        else:
            l=m//g*n; k=((b-a)//g*x)%(n//g); out.append(f"{(a+m*k)%l} {l}")
    print("\n".join(out))
if __name__=="__main__": main()
```

4 C. Fast Fibonacci

4.1 Restatement

Answer Fibonacci queries modulo `1e9+7`.

4.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. □

4.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

4.4 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std; const long long MOD=1000000007; pair<long long,long long> fib(long long
```

4.5 Full Python solution

```python
import sys
MOD=1_000_000_007
def fib(n):
    if n==0: return (0,1)
    a,b=fib(n//2)
    c=a*((2*b-a)%MOD)%MOD
    d=(a*a+b*b)%MOD
    return (d,(c+d)%MOD) if n%2 else (c,d)
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    print("\n".join(str(fib(x)[0]) for x in data[1:1+data[0]]))
if __name__=="__main__": main()
```

5 D. Fibonacci Range Sum

5.1 Restatement and observation

For a huge inclusive interval `[l,r]`, sum Fibonacci values modulo `MOD`. The blind step is recognizing the telescoping identity:

```
F(0)+F(1)+...+F(t) = F(t+2)-1.
```

Therefore the requested result is:

```
F(r+2) - F(l+1) mod MOD.
```

5.2 Algorithm

Use fast doubling to obtain `(F(n),F(n+1))` in `O(log n)`:

```
F(2k)   = F(k) * (2F(k+1)-F(k))
F(2k+1) = F(k)^2 + F(k+1)^2.
```

Apply it twice per query and normalize the subtraction.

5.3 Correctness proof

The prefix identity follows by induction from `F(i+2)=F(i+1)+F(i)`. Subtracting the prefix through `l-1` from the prefix through `r` gives exactly `[l,r]`. Fast doubling identities compute the required Fibonacci values by splitting the index in half, so the printed normalized difference is correct. □

5.4 Complexity and implementation

Each query costs `O(log r)` time and `O(log r)` recursion stack; values are always reduced modulo `1,000,000,007`. Use unsigned 64-bit indices in C++ for the `10^18` bound and Python integers in Python.

5.5 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std; using ll=long long; const ll MOD=1000000007;
pair<ll,ll> fib(unsigned long long n){
    if (!n) return {0, 1};
    auto [a, b] = fib(n / 2);
    ll c=a*((2*b%MOD-a+MOD)%MOD)%MOD, d=(a*a%MOD+b*b%MOD)%MOD;
    return n&1?make_pair(d,(c+d)%MOD):make_pair(c,d);
}
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int q;cin>>q;while(q--){unsigned lor
```

5.6 Full Python solution

```python
import sys
MOD=1_000_000_007
def fib(n):
    if n==0:return 0,1
    a,b=fib(n//2); c=a*((2*b-a)%MOD)%MOD; d=(a*a+b*b)%MOD
    return (d,(c+d)%MOD) if n&1 else (c,d)
def main():
    data=list(map(int,sys.stdin.buffer.read().split())); out=[]
    for i in range(1,len(data),2): out.append(str((fib(data[i+1]+2)[0]-fib(data[i]+1)[0])%MO
    print('\n'.join(out))
if __name__=='__main__':main()
```

5.7 Tests that matter

```
[0,0], [0,1], one-element ranges, and a range with l>0.
Large endpoints: never iterate through the interval.
```

6 E. Pattern Prefix Matches

6.1 From many comparisons to one Z-function

At every text position `i`, we need the longest prefix of `pattern` equal to a prefix of `text[i..]`. Comparing directly at every position is quadratic on inputs such as repeated `a` characters.

Construct:

```
combined = pattern + '#' + text
```

where `#` is absent from the lowercase input alphabet. The Z-value `z[j]` is the longest prefix of `combined` matching `combined[j..]`. For text position `i`, index `|pattern| + 1 + i` therefore compares exactly the pattern prefix with the text starting at `i`.

6.2 Computing the Z-array

Maintain the rightmost interval `[l,r]` already known to equal a prefix. For a new index `i`:

- if `i > r`, start comparing from zero;
- otherwise reuse `min(r-i+1, z[i-l])` known matching characters;
- extend with direct comparisons;
- update `[l,r]` if this match reaches farther.

The separator ensures a match originating in the text cannot continue through the boundary into the text again. Capping the extracted Z-value by the pattern length makes the intended limit explicit.

6.3 Correctness proof

By definition of the Z-function, the value at combined index `|pattern|+1+i` is the longest prefix of `combined` matching from text position `i`. The prefix of `combined` begins with `pattern`. Because the separator does not occur in either input string, equality cannot pass its position, so the matched characters are exactly a prefix of `pattern`. Therefore the extracted value is precisely the requested match length.

The standard Z-box algorithm computes the same definition: reused characters lie inside a previously proven prefix-equal interval, and the extension loop checks every additional character explicitly. Hence all reported values are correct.

6.4 Complexity and pitfalls

The right boundary advances at most the combined-string length, so total time and memory are `O(|pattern|+|text|)`.

Choose a separator outside the alphabet, offset text indices by `|pattern|+1`, and output one value for every text position – including zeros. A naive restart of comparisons at every position does not meet the bound.

6.5 Tests that matter

```
Pattern longer than text.
No matching first character anywhere.
Text equal to the pattern.
Highly periodic strings such as aaaa and aaaaaa.
Overlapping matches such as pattern aba in abacaba.
Matches ending exactly at the end of text.
```

For small strings, compare characters directly at each text position.

6.6 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string p,t;cin>>p>>t;string s=p+"#"+
```

6.7 Full Python solution

```python
import sys
def main():
    p,t=sys.stdin.read().split();s=p+"#"+t;z=[0]*len(s);l=r=0
    for i in range(1,len(s)):
        if i<=r:z[i]=min(r-i+1,z[i-l])
        while i+z[i]<len(s) and s[z[i]]==s[i+z[i]]:z[i]+=1
        if i+z[i]-1>r:l,r=i,i+z[i]-1
    print(" ".join(str(min(len(p),z[len(p)+1+i])) for i in range(len(t))))
if __name__=="__main__":main()
```

7 F. Suffix LCP Queries

7.1 Restatement and monotone predicate

For suffixes starting at `i` and `j`, find their longest common prefix. If their first `L` characters are equal, then every shorter prefix is also equal; if they differ within the first `L`, every longer prefix differs as well. Prefix equality is therefore a monotone predicate suitable for binary search.

The remaining requirement is an `O(1)` substring-equality test. Polynomial rolling hashes provide it after linear preprocessing.

7.2 Double substring hashing

For each of two moduli, precompute powers of a base and prefix hashes:

```
H[r] = hash of s[0..r)
hash(l,r) = H[r] - H[l] * base^(r-l) mod modulus
```

Equal substrings have equal hashes. Using two independent large prime moduli makes an accidental collision negligibly likely.

For query `(i,j)`, the answer is between zero and `n - max(i,j)`. Binary-search the largest length `L` for which the two hash pairs of `s[i..i+L)` and `s[j..j+L)` agree.

7.3 Correctness argument

Ignoring the negligible possibility of a hash collision, the hash predicate is true exactly when the two length-`L` substrings are equal. Equality of length `L` implies equality for every smaller length, so upper-bound binary search returns the greatest true `L`. That value is the definition of the LCP of the two suffixes.

The search upper bound prevents either substring from passing the end of the string. In particular, querying a suffix against itself correctly returns its full remaining length.

7.4 Complexity and alternatives

Power and prefix tables take `O(n)` time and memory. Each query performs `O(log n)` hash comparisons, each `O(1)`, for total `O(n + q log n)` time.

This intended solution is probabilistic. A suffix array with an LCP array and RMQ can answer deterministically, but requires more machinery. Common hash errors are mixing inclusive and half-open endpoints, failing to normalize a negative remainder, or using powers for the wrong substring length.

7.5 Tests that matter

```
i = j                          -> entire remaining suffix
Different first characters   -> 0
One suffix is a prefix of the other
Repeated string aaaaa        -> long LCPs at nearby positions
banana queries               -> overlapping periodic suffixes
Positions near n             -> upper-bound correctness
```

The randomized oracle compares the two suffixes character by character.

7.6 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;using ll=long long;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;cin>>s;int n=s.size();const
```

7.7 Full Python solution

```python
import sys
def main():
    d=sys.stdin.buffer.read().split();s=d[0];n=len(s);q=int(d[1]);mods=(1_000_000_007,1_000_
    powers=[];prefix=[]
    for mod in mods:
        p=[1]*(n+1);h=[0]*(n+1)
        for i,c in enumerate(s):p[i+1]=p[i]*base%mod;h[i+1]=(h[i]*base+c)%mod
        powers.append(p);prefix.append(h)
    def equal(a,b,length):
        for mod,p,h in zip(mods,powers,prefix):
            if (h[a+length]-h[a]*p[length])%mod!=(h[b+length]-h[b]*p[length])%mod:return Fal
        return True
    out=[];idx=2
    for _ in range(q):
        a,b=int(d[idx])-1,int(d[idx+1])-1;idx+=2;lo,hi=0,n-max(a,b)
        while lo<hi:
            mid=(lo+hi+1)//2
            if equal(a,b,mid):lo=mid
            else:hi=mid-1
        out.append(str(lo))
    print("\n".join(out))
if __name__=="__main__":main()
```

8 G. Domino Placement

8.1 Reduction to bipartite matching

Color grid cells by parity of `row + column`. Cardinal neighbors always have opposite parity. Create one graph vertex for each open cell and an edge for each cardinally adjacent pair, placing even cells on the left and odd cells on the right.

A domino covers exactly one such graph edge. Requiring dominoes not to overlap means no two chosen edges may share a vertex – which is precisely a matching. Thus the geometry disappears after the reduction.

8.2 Augmenting-path algorithm

Maintain `match[v]`, the left cell currently paired with right-side cell `v`. For every open left cell, search for an augmenting path:

1. Try each adjacent right cell.
2. If it is free, match it immediately.
3. If it is occupied, recursively try to reroute its current left partner.
4. On success, assign the right cell to the current left cell.

Use a fresh visitation stamp for each search so a right vertex is explored at most once during that attempt.

8.3 Correctness proof

There is a bijection between legal domino placements and matchings: every domino maps to its adjacency edge, and non-overlap maps to disjoint endpoints; every matching maps back to non-overlapping adjacent dominoes.

The augmenting-path procedure starts with an empty matching and increases its size by one whenever it finds an alternating path from an unmatched left vertex to an unmatched right vertex. When no such path exists for any left vertex, Berge's lemma says the matching is maximum. Therefore the matching size, and hence the printed number of dominoes, is optimal.

8.4 Complexity and implementation notes

Let `V` be the number of open cells and `E <= 4V` the adjacency edges. The simple Kuhn algorithm runs in `O(VE)` worst-case time and uses `O(V+E)` memory, which is sufficient for a `100 x 100` grid. Hopcroft–Karp would improve the bound to `O(E sqrt(V))` if larger grids were required.

Only start searches from one parity; otherwise each grid adjacency is treated from both sides and the matching logic is invalid. Walls create no graph vertex. In Python, recursive augmenting paths require a sufficiently generous recursion limit or an iterative implementation.

8.5 Tests that matter

```
All walls                          -> 0
One open cell                      -> 0
Two adjacent versus diagonal cells
A corridor of odd length           -> floor(length/2)
Disconnected open regions
A board where greedy local pairing blocks a larger placement
```

For tiny grids, enumerate subsets of adjacency edges and keep the largest vertex-disjoint set.

8.6 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m;cin>>n>>m;vector<string>g(n)
```

8.7 Full Python solution

```python
import sys
def main():
    data=sys.stdin.read().split();n,m=map(int,data[:2]);g=data[2:];match=[-1]*(n*m);seen=[0]
    def dfs(cell):
        if seen[cell]==stamp:return False
        seen[cell]=stamp;r,c=divmod(cell,m)
        for dr,dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr,nc=r+dr,c+dc
            if 0<=nr<n and 0<=nc<m and g[nr][nc]==".":
                v=nr*m+nc
                if match[v]==-1 or dfs(match[v]):match[v]=cell;return True
        return False
    ans=0
    for r in range(n):
        for c in range(m):
```

```
            if g[r][c]=="." and (r+c)%2==0:
                stamp+=1;ans+=dfs(r*m+c)
    print(ans)
if __name__=="__main__":main()
```

9 H. Weighted Tree Distances

9.1 Restatement and why rerooting is needed

For every possible root `r`, compute:

```
answer[r] = sum over v of weight[v] * distance(r,v)
```

A BFS or DFS from every root is `O(n^2)`. Neighboring roots have closely related answers, so compute one root directly and transfer its value across each edge.

9.2 First pass: root at vertex 1

An iterative traversal records `parent`, `depth`, and an order. Then:

```
answer[1] = sum(weight[v] * depth[v])
subtree[u] = sum of weights in u's rooted subtree
```

Compute subtree weights in reverse traversal order.

9.3 Reroot transition

Move the root from parent `u` to child `v`. Distances to vertices in `v`'s subtree decrease by one, changing the sum by `-subtree[v]`. Distances to all other vertices increase by one, changing it by `total_weight - subtree[v]`. Therefore:

```
answer[v] = answer[u] + total_weight - 2 * subtree[v]
```

Applying this formula in parent-before-child order obtains every answer.

9.4 Correctness proof

The first answer is correct by the definition of depth from root `1`. Reverse-order accumulation gives each subtree's exact total weight because every child's completed sum is added once to its parent.

For an edge from `u` to child `v`, the edge separates the tree into exactly two sides. Rerooting at `v` shortens every path to the `v` side by one and lengthens every path to the other side by one. The derived formula accounts for the total weight on both sides exactly. By induction along the rooted tree, if `answer[u]` is correct then `answer[v]` is correct; since the root answer is correct, all answers are correct.

9.5 Complexity and pitfalls

Both traversals examine every edge a constant number of times, giving `O(n)` time and `O(n)` memory. Use 64-bit integers for weights, subtree sums, and answers. Iterative traversal avoids recursion overflow at `n = 200000`.

Do not use subtree vertex counts in the formula – the values are weighted. Zero-weight vertices remain structurally present even though they add nothing to a sum.

9.6 Tests that matter

```
One vertex.
Two vertices with unequal weights.
A path, checking both endpoints and a middle root.
A star with a heavy leaf.
All weights zero.
Large weights on a deep path -> overflow check.
```

For small trees, run BFS from every root and evaluate the definition directly.

9.7 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;using ll=long long;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;cin>>n;vector<ll>w(n);for(ll&x
```

9.8 Full Python solution

```python
import sys
def main():
    d=list(map(int,sys.stdin.buffer.read().split()));n=d[0];w=d[1:n+1];g=[[]for _ in range(n
    for _ in range(n-1):u,v=d[idx]-1,d[idx+1]-1;idx+=2;g[u].append(v);g[v].append(u)
    par=[-1]*n;depth=[0]*n;order=[0]
    for u in order:
        for v in g[u]:
            if v!=par[u]:par[v]=u;depth[v]=depth[u]+1;order.append(v)
    sub=w[:]
    for v in reversed(order[1:]):sub[par[v]]+=sub[v]
    total=sum(w);ans=[0]*n;ans[0]=sum(w[i]*depth[i] for i in range(n))
    for u in order:
        for v in g[u]:
            if par[v]==u:ans[v]=ans[u]+total-2*sub[v]
    print(" ".join(map(str,ans)))
if __name__=="__main__":main()
```

10 I. Balanced Subset Difference

10.1 Reformulation

Putting a subset into the first group determines the other group. If the total sum is `T` and the chosen subset sum is `S`, the absolute group difference is:

```
|S - (T-S)| = |T - 2S|
```

We need a subset sum as close as possible to `T/2`. Enumerating all `2^40` subsets is too large, but splitting the values into two halves reduces each enumeration to about `2^20`.

Signed values do not invalidate the reformulation; they only mean subset sums are not naturally ordered until we sort them.

10.2 Meet-in-the-middle algorithm

1. Split the input into left and right halves.
2. Enumerate all subset sums `L` of the left half and all subset sums `R` of the right half, including the empty subset.
3. Sort `R`.
4. For every `x` in `L`, seek a `y` in `R` making `x+y` closest to `T/2`. Binary-search the insertion position for that target and test the neighbor on each side.
5. Minimize `|T - 2(x+y)|`.

Every subset of the original array decomposes uniquely into one left subset and one right subset.

10.3 Correctness proof

Let an optimal full subset have half-sums `(x,y)`. The algorithm enumerates its left sum `x`. For this fixed `x`, the objective as a function of `y` is the absolute distance from a target, so among sorted right sums its minimum occurs at the first value not below the target or its predecessor. The algorithm checks both. It therefore finds a right sum at least as good as the optimal subset's `y`. Since every checked pair also corresponds to a valid full subset, the global minimum is exactly optimal.

10.4 Complexity and implementation notes

Each half has at most `2^20` sums. Enumeration is `O(2^(n/2))`; sorting and binary searches give `O(2^(n/2) log 2^(n/2))` time and `O(2^(n/2))` memory.

Use 64-bit signed sums. Always include the empty subset because either group may be empty. With negative totals, avoid a careless integer division by two; equivalently binary-search an integer threshold derived from `|T - 2(x+y)|` and check both adjacent values.

10.5 Tests that matter

```
n = 0 or one value, if permitted by the contract.
An exact balanced split -> 0.
All values positive with no exact split.
Mixed positive and negative values.
All negative values.
An optimum using the empty subset or all values.
Values near 10^12 -> 64-bit sums.
```

For small `n`, enumerate every full subset and evaluate `|T-2S|`.

10.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;using ll=long long;
vector<ll>sums(const vector<ll>&a){vector<ll>s{0};for(ll x:a){int n=s.size();for(int i=0;i<n
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;cin>>n;vector<ll>a(n);for(ll&x
```

10.7 Full Python solution

```
import bisect,sys
def sums(a):
    out=[0]
    for x in a:out += [v+x for v in out]
    return out
def main():
    d=list(map(int,sys.stdin.buffer.read().split()));n=d[0];a=d[1:];left=sums(a[:n//2]);righ
    for x in left:
        target=(total-2*x+1)//2;i=bisect.bisect_left(right,target)
        for j in (i-1,i):
            if 0<=j<len(right):best=min(best,abs(total-2*(x+right[j])))
    print(best)
if __name__=="__main__":main()
```

11 J. Monge Partition Table

11.1 Baseline interval-partition DP

Partition positions `1..n` into exactly `k` nonempty contiguous groups. Define:

```
dp[g][i] = minimum cost to partition positions 1..i into g groups
```

If the last group is `[p,i]`, the previous `g-1` groups cover `1..p-1`:

```
dp[g][i] = min over g <= p <= i of
           dp[g-1][p-1] + cost[p][i]
```

The straightforward evaluation is `O(kn^2)`, too slow for the stated bounds.

11.2 Monotone optima and divide-and-conquer

The input promises a Monge interval-cost table. The standard monotonicity theorem for this recurrence then guarantees:

```
opt[g][i] <= opt[g][i+1]
```

where `opt[g][i]` is a minimizing start `p` for the last group.

To compute one DP layer on output interval `[lo,hi]`, evaluate its midpoint `mid`, searching only candidate splits `[optL,optR]`. If `best` minimizes the midpoint, recursively compute:

```
left outputs  [lo,mid-1] with candidates [optL,best]
right outputs [mid+1,hi] with candidates [best,optR]
```

Monotonicity proves that no discarded candidate can be optimal in either half.

11.3 Correctness proof

The baseline recurrence is exact because every valid partition has a unique start `p` of its last group, and combining an optimal prefix partition with `[p,i]` constructs a valid candidate.

For the optimized evaluation, assume the recursive call's candidate interval contains the optimum for every output index it owns. The midpoint search finds its true optimum `best`. Monotonicity places every optimum to the left at or before `best`, and every optimum to the right at or after `best`, so the two recursive candidate ranges preserve the assumption. Induction over recursive calls shows every computed DP value equals the baseline recurrence, and hence `dp[k][n]` is optimal.

11.4 Complexity and implementation notes

At one recursion depth, the searched candidate ranges have total length `O(n)`; there are `O(log n)` depths. Each of `k` layers therefore takes `O(n log n)`, for `O(kn log n)` time. DP layers use `O(n)` memory, while the given cost table itself uses `O(n^2)`.

Exactly `g` nonempty groups means layer `g` starts at position `i = g`. Initialize only `dp[0][0] = 0`; impossible states stay at infinity. Be precise when translating one-based groups into the zero-based table `cost[p-1][i-1]`.

11.5 Tests that matter

```
k = 1                          -> cost[1][n]
k = n                          -> sum of singleton costs
Small table checked by O(kn^2) DP
Several tied optimal splits    -> monotonic tie handling
Large costs                    -> infinity and overflow safety
```

Random tests should generate a table known to be Monge and compare against the quadratic recurrence, rather than assuming the optimized code is its own oracle.

11.6 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;using ll=long long;const ll INF=(1LL<<62);
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,k;cin>>n>>k;vector<vector<ll>>
```

11.7 Full Python solution

```python
import sys
def main():
    d=list(map(int,sys.stdin.buffer.read().split()));n,k=d[:2];v=d[2:];cost=[v[i*n:(i+1)*n]f
    for group in range(1,k+1):
        cur=[inf]*(n+1)
        def solve(lo,hi,optl,optr):
            if lo>hi:return
            mid=(lo+hi)//2;best=optl
            for j in range(optl,min(optr,mid-1)+1):
                value=prev[j]+cost[j][mid-1]
                if value<cur[mid]:cur[mid]=value;best=j
            solve(lo,mid-1,optl,best);solve(mid+1,hi,best,optr)
        solve(group,n,group-1,n-1);prev=cur
    print(prev[n])
if __name__=="__main__":main()
```

12 K. Online Line DP

12.1 Turning the recurrence into line queries

The transition is:

```
dp[i] = fee[i] + min over j < i of
        (dp[j] + slope[j] * x[i])
```

For a fixed earlier index `j`, the expression depending on `x[i]` is a line:

```
y = slope[j] * x + dp[j]
```

Thus each step queries the minimum of all previous lines at coordinate `x[i]`, adds `fee[i]`, then inserts the new line from index `i`. The insertions must be online because line `i` contains the just-computed `dp[i]`.

No monotonicity is promised for slopes or query coordinates, so a monotone CHT deque is not valid. A Li Chao tree supports arbitrary order.

12.2 Li Chao invariant

Build the tree over the sorted distinct coordinates appearing in `x`. Each node stores one line that is best at that node's midpoint among lines compared there. When inserting a new line:

1. At the midpoint, keep the better line in the node.
2. The displaced line can beat the stored line on at most one side because two lines cross at most once.
3. Compare at the interval endpoints and recurse only into the side where the displaced line can still win.

A query follows the root-to-leaf path for its compressed coordinate and takes the minimum of every stored line encountered.

12.3 Correctness proof

The insertion rule never discards a line from a region where it might be optimal: after keeping the midpoint winner, the other line can only become better on one side, and it is recursively stored on precisely that side. Inductively, for every inserted line and every coordinate, either the line is stored on the query path or another stored line is no worse there. Therefore a query returns the minimum over all inserted lines.

Before computing `dp[i]`, the inserted lines correspond exactly to indices `j < i`. The query consequently returns the minimum transition term, and adding `fee[i]` gives the recurrence. Inserting line `i` restores the invariant for the next iteration. By induction, the final `dp[n-1]` is correct.

12.4 Complexity and pitfalls

There are at most `n` compressed coordinates. Each insertion and query descends `O(log n)` levels, for `O(n log n)` time and `O(n)` tree memory.

- Insert line `0` with intercept `dp[0] = 0` before the first query.
- Query before inserting line `i`; otherwise the forbidden choice `j = i` becomes available.
- Coordinate compression changes only the domain, not the actual `x` used in line evaluation.
- Use 64-bit arithmetic, or wider intermediates if the contract permits products near the type limit.
- Duplicate slopes and duplicate coordinates must be supported.

12.5 Tests that matter

```
n = 1                          -> dp[0] = 0
One transition                 -> direct formula
Unsorted and duplicate x coordinates
Increasing, decreasing, and duplicate slopes
A newly inserted line wins only on one extreme
Negative slopes and coordinates
Large products                 -> integer-width check
```

The small oracle evaluates every earlier `j` directly in `O(n^2)`.

12.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;using ll=long long;const ll INF=(1LL<<62);struct Line{ll m=0,b=INF;ll at
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;cin>>n;vector<ll>x(n),m(n),fee
```

12.7 Full Python solution

```
import sys
def main():
    d=list(map(int,sys.stdin.buffer.read().split()));n=d[0];x=d[1:n+1];m=d[n+1:2*n+1];fee=d[
    def value(line,at):return line[0]*at+line[1]
    def add(line,p,l,r):
```

```
        mid=(l+r)//2
        if value(tree[p],coords[mid])>value(line,coords[mid]):tree[p],line=line,tree[p]
        if l==r:return
        if value(tree[p],coords[l])>value(line,coords[l]):add(line,p*2,l,mid)
        elif value(tree[p],coords[r])>value(line,coords[r]):add(line,p*2+1,mid+1,r)
    def get(p,l,r,at):
        ans=value(tree[p],coords[at])
        if l==r:return ans
        mid=(l+r)//2
        return min(ans,get(p*2,l,mid,at)if at<=mid else get(p*2+1,mid+1,r,at))
    dp=[0]*n;add((m[0],0),1,0,len(coords)-1)
    for i in range(1,n):
        at=__import__("bisect").bisect_left(coords,x[i]);dp[i]=fee[i]+get(1,0,len(coords)-1,
    print(dp[-1])
if __name__=="__main__":main()
```

13 L. Profitable Disjoint Routes

13.1 Reduction to min-cost flow

Give every original edge capacity `1` and cost equal to the negative of its profit. Send exactly `k` units of flow from `1` to `n`. Integral unit-capacity flow decomposes into `k` edge-disjoint source-to-sink paths (cycles can be discarded), and any such path collection gives a flow of the same total profit. Thus maximizing profit is exactly minimizing flow cost.

Use the residual graph. Repeatedly find a shortest source-to-sink residual path, augment one unit, and add its cost. Negative original costs are allowed; the reference uses a queue-based Bellman–Ford relaxation. If no augmenting path exists before `k` units, the requested collection is impossible.

13.2 Correctness proof

The reduction above is value-preserving in both directions. For the algorithm, after an optimal flow of value `f`, a cheapest residual augmenting path produces an optimal flow of value `f+1`: if a cheaper value-`f+1` flow existed, subtracting the current flow would decompose into residual cycles and one net source-to-sink augmentation with smaller total residual cost. That contradicts the chosen shortest augmentation (nonnegative-cost residual cycles can be discarded). Induction proves optimality at flow `k`.

13.3 Complexity and tests

With queue relaxation, the conservative bound is `O(kVE)` time and `O(V+E)` memory. Test `k=1`, exactly `k` outgoing source edges, fewer than `k` paths, negative profits, and a case where a later residual reverse edge changes an earlier routing choice.

13.4 Full solutions

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct E {
  int to, rev, cap;
  ll cost;
};
int main() {
  ios::sync_with_stdio(false);
```

```
cin.tie(nullptr);
int n, m, k;
if (!(cin >> n >> m >> k))
  return 0;
vector<vector<E>> g(n);
auto add = [&](int u, int v, ll c) {
  g[u].push_back({v, (int)g[v].size(), 1, c});
  g[v].push_back({u, (int)g[u].size() - 1, 0, -c});
};
while (m--) {
  int u, v;
  ll p;
  cin >> u >> v >> p;
  add(u - 1, v - 1, -p);
}
ll cost = 0;
int flow = 0;
const ll INF = 4e18;
while (flow < k) {
  vector<ll> d(n, INF);
  vector<int> pv(n), pe(n);
  vector<char> in(n);
  queue<int> q;
  d[0] = 0;
  q.push(0);
  in[0] = 1;
  while (!q.empty()) {
    int u = q.front();
    q.pop();
    in[u] = 0;
    for (int i = 0; i < (int)g[u].size(); i++) {
      E &e = g[u][i];
      if (e.cap && d[e.to] > d[u] + e.cost) {
        d[e.to] = d[u] + e.cost;
        pv[e.to] = u;
        pe[e.to] = i;
        if (!in[e.to])
          in[e.to] = 1, q.push(e.to);
      }
    }
  }
  if (d[n - 1] == INF)
    break;
  cost += d[n - 1];
  flow++;
  for (int v = n - 1; v; v = pv[v]) {
    E &e = g[pv[v]][pe[v]];
    e.cap--;
    g[v][e.rev].cap++;
  }
}
```

```
  if (flow < k)
    cout << "IMPOSSIBLE\n";
  else
    cout << -cost << '\n';
}
```

```
from collections import deque
import sys
def main():
    d=list(map(int,sys.stdin.buffer.read().split()))
    if not d:return
    n,m,k=d[:3];g=[[]for _ in range(n)];j=3
    def add(u,v,c):
        g[u].append([v,len(g[v]),1,c]);g[v].append([u,len(g[u])-1,0,-c])
    for _ in range(m):u,v,p=d[j:j+3];j+=3;add(u-1,v-1,-p)
    flow=0;cost=0;inf=10**30
    while flow<k:
        dist=[inf]*n;pv=[-1]*n;pe=[-1]*n;inside=[False]*n;dist[0]=0;q=deque([0]);inside[0]=T
        while q:
            u=q.popleft();inside[u]=False
            for i,e in enumerate(g[u]):
                v,rev,cap,c=e
                if cap and dist[v]>dist[u]+c:
                    dist[v]=dist[u]+c;pv[v]=u;pe[v]=i
                    if not inside[v]:inside[v]=True;q.append(v)
        if dist[-1]==inf:break
        flow+=1;cost+=dist[-1];v=n-1
        while v:
            u=pv[v];i=pe[v];rev=g[u][i][1];g[u][i][2]-=1;g[v][rev][2]+=1;v=u
    print(-cost if flow==k else 'IMPOSSIBLE')
if __name__=="__main__":main()
```

14 M. Dial Alignment

14.1 Restatement and constraints

Count offsets, rather than distinct rotated strings. Both strings have the same length `n <= 200000`; constructing and comparing each complete rotation would take `O(n^2)` time.

14.2 Observation and algorithm

Every rotation of `s` is a length-`n` window of `s+s` starting at an index in `0..n-1`. Therefore count occurrences of `t` in `(s+s)[:-1]` using KMP. Omitting the final character removes the duplicated window starting at `n`.

The prefix function of `t` stores, for each prefix, its longest proper border. During the scan, `matched` is the longest pattern prefix equal to a suffix of the scanned text. On a mismatch, follow prefix-function borders until the next character fits or the length reaches zero. After a full match, keep the longest proper border so overlapping occurrences remain available.

```
compute the prefix function of t
scan the first 2*n-1 characters of s+s with KMP
count every complete match, retaining its longest proper border
```

14.3 Sample walkthrough

For `s = abab`, scan `abababa` for `t = baba`. The complete matches start at indices `1` and `3`, representing two dial offsets. Searching all of `s+s` would also duplicate offset zero when that offset is a match, for example when `s = t = abab`.

14.4 Correctness proof

There is a bijection between valid offsets and pattern matches in the scanned text: offset `k` reads exactly its length-`n` window, and all possible window starts in a length-`2n-1` text are exactly `0..n-1`. KMP's invariant preserves the longest matching prefix after every character. Its border fallbacks remove only prefix lengths already contradicted by the mismatch. It consequently reports exactly all full occurrences, including overlapping ones. Their count is precisely the number of valid offsets.

14.5 Complexity and language notes

Prefix construction and scanning take `O(n)` time and `O(n)` memory. Both references use 0-based prefix lengths and reset `matched` immediately after a full match, before any further access to `target[matched]`. Python constructs one doubled string, not a fresh rotation for each offset. C++ uses integer lengths, which fit comfortably at this bound.

14.6 Common mistakes and targeted tests

Do not deduplicate rotations, skip overlaps, or count the repeated offset `n`. Fixed cases include single characters, no match, and a periodic string. The random oracle builds every small rotation explicitly. Maximum-size tests use all equal letters (every offset works) and one exceptional final letter (only one offset works, with long unsuccessful comparisons for direct methods).

14.7 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string source, target;
    cin >> source >> target;
    int n = target.size();
    vector<int> prefix(n);
    for (int i = 1; i < n; ++i) {
        int j = prefix[i - 1];
        while (j && target[i] != target[j]) j = prefix[j - 1];
        if (target[i] == target[j]) ++j;
        prefix[i] = j;
    }
    string doubled = source + source;
    int answer = 0, matched = 0;
    for (int i = 0; i < 2 * n - 1; ++i) {
        char ch = doubled[i];
        while (matched && ch != target[matched])
            matched = prefix[matched - 1];
        if (ch == target[matched]) ++matched;
        if (matched == n) {
            ++answer;
            matched = prefix[matched - 1];
        }
    }
    cout << answer << '\n';
}
```

14.8 Full Python solution

```
import sys


def count_alignments(source, target):
    n = len(target)
    prefix = [0] * n
    for i in range(1, n):
        j = prefix[i - 1]
        while j and target[i] != target[j]:
            j = prefix[j - 1]
        if target[i] == target[j]:
            j += 1
        prefix[i] = j

    answer = matched = 0
    # Exactly n possible starting positions; omit the repeated last window.
    for ch in (source + source)[:-1]:
        while matched and ch != target[matched]:
            matched = prefix[matched - 1]
        if ch == target[matched]:
            matched += 1
        if matched == n:
            answer += 1
            matched = prefix[matched - 1]
    return answer


def main():
    source, target = sys.stdin.read().split()
    print(count_alignments(source, target))


if __name__ == '__main__':
    main()
```

15 N. Archive Reassembly

15.1 Restatement and constraints

Partition the target into the fewest nonempty pieces, each a substring of the source. Both lengths can reach `200000`. Pieces may be reused without limit, so source occurrences are permissions, not consumable resources.

15.2 Longest feasible next piece

Starting at the current target position, take the longest prefix that occurs in the source. A suffix automaton of the source recognizes exactly its substrings: a path from state zero exists if and only if its letters form an allowed piece. Every visited state is sufficient; terminal-suffix flags are not needed for substring membership.

Walk transitions while possible. If the next character has no transition, commit the current piece and retry that character from state zero. If even state zero lacks its character, no piece can contain it, so return `-1`. The references start with one prospective piece because the target is nonempty.

```
build the source suffix automaton
pieces = 1; state = 0
for each target character:
    if its transition from state is missing:
        pieces += 1
        restart from state 0
        if the root transition is missing: answer -1
    follow the transition
answer pieces
```

15.3 Sample walkthrough

For source `abac` and target `acaba`, the longest first piece is `ac`. There is no transition extending this path with the next `a`, so restart at that `a`. The remaining `aba` is a source substring. The answer is `2`. Repeated copies and overlapping source locations are legal.

15.4 Correctness proof of the greedy choice

Every substring of an allowed piece is itself allowed. Let a greedy first piece end at position `g`, and consider an optimal partition of the same target suffix. Its first piece cannot extend beyond `g`, by maximality of the greedy match. Find the optimal piece containing position `g`.

Replace all optimal pieces through that point by the greedy piece, retaining the part after `g` of the containing piece if it is nonempty.

That retained part is still a source substring. If only one optimal piece was replaced, it already ended at `g`, so no extra piece is introduced. If several were replaced, one greedy piece plus at most one retained suffix uses no more pieces than before. Thus an optimal partition can be made to start with the greedy piece. Induction on the remaining target proves optimality.

If the root lacks a character, that character occurs nowhere in the source, so no partition is possible. Otherwise single-character pieces always give a construction, and the greedy algorithm returns its minimum piece count.

15.5 Automaton reminder and complexity

An ordinary new state represents the new longest prefix. Suffix links visit its shorter suffix groups. Missing character transitions are added along that chain; an existing destination whose length range is too wide is split by a clone, preserving its transitions and redirecting the relevant incoming edges. This is the construction taught in section 44.

For the fixed lowercase alphabet, construction takes `O(|s|)` time and memory. Each target character requires at most one failed lookup and one root lookup, so total time is `O(|s|+|t|)`. C++ uses arrays of 26 transitions and integer state IDs; Python uses dictionaries and parallel state arrays. A clone copies its transition dictionary rather than aliasing it. Neither reference uses recursion or enumerates source substrings.

15.6 Common mistakes and targeted tests

Do not consume source positions, require pieces to be suffixes, or forget to retry the character that caused a restart. Fixed cases test absent characters, reusing a one-letter source, and paths through cloned states. The random oracle enumerates all small source substrings and uses an exhaustive partition DP. Full-size cases include two long alternating pieces and a clone-heavy random source copied whole, whose expected answer is immediately `1`.

15.7 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

struct SuffixAutomaton {
    struct State {
        array<int, 26> next;
        int length = 0, link = -1;
        State() { next.fill(-1); }
    };
    vector<State> states;
    int last = 0;

    explicit SuffixAutomaton(const string& text) {
```

```
        states.reserve(2 * text.size());
        states.emplace_back();
        for (char ch : text) extend(ch - 'a');
    }

    void extend(int ch) {
        int cur = states.size();
        states.emplace_back();
        states[cur].length = states[last].length + 1;
        int p = last;
        while (p != -1 && states[p].next[ch] == -1) {
            states[p].next[ch] = cur;
            p = states[p].link;
        }
        if (p == -1) {
            states[cur].link = 0;
        } else {
            int q = states[p].next[ch];
            if (states[p].length + 1 == states[q].length) {
                states[cur].link = q;
            } else {
                int clone = states.size();
                states.push_back(states[q]);
                states[clone].length = states[p].length + 1;
                while (p != -1 && states[p].next[ch] == q) {
                    states[p].next[ch] = clone;
                    p = states[p].link;
                }
                states[q].link = states[cur].link = clone;
            }
        }
        last = cur;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string source, target;
    cin >> source >> target;
    SuffixAutomaton automaton(source);
    int state = 0, pieces = 1;
    for (char letter : target) {
        int ch = letter - 'a';
        int to = automaton.states[state].next[ch];
        if (to == -1) {
            ++pieces;
            to = automaton.states[0].next[ch];
            if (to == -1) {
                cout << -1 << '\n';
                return 0;
```

```
            }
        }
        state = to;
    }
    cout << pieces << '\n';
}
```

15.8 Full Python solution

```
import sys


class SuffixAutomaton:
    def __init__(self, text):
        self.next = [{}]
        self.length = [0]
        self.link = [-1]
        last = 0
        for ch in text:
            cur = len(self.next)
            self.next.append({})
            self.length.append(self.length[last] + 1)
            self.link.append(0)
            p = last
            while p != -1 and ch not in self.next[p]:
                self.next[p][ch] = cur
                p = self.link[p]
            if p != -1:
                q = self.next[p][ch]
                if self.length[p] + 1 == self.length[q]:
                    self.link[cur] = q
                else:
                    clone = len(self.next)
                    self.next.append(self.next[q].copy())
                    self.length.append(self.length[p] + 1)
                    self.link.append(self.link[q])
                    while p != -1 and self.next[p].get(ch) == q:
                        self.next[p][ch] = clone
                        p = self.link[p]
                    self.link[q] = self.link[cur] = clone
            last = cur


def minimum_pieces(source, target):
    automaton = SuffixAutomaton(source)
    state = 0
    pieces = 1
    for ch in target:
```

```
        to = automaton.next[state].get(ch)
        if to is None:
            # Commit the longest possible piece, then retry from the root.
            pieces += 1
            to = automaton.next[0].get(ch)
            if to is None:
                return -1
        state = to
    return pieces


def main():
    source, target = sys.stdin.read().split()
    print(minimum_pieces(source, target))


if __name__ == '__main__':
    main()
```

16 O. Relay Throughput

16.1 Restatement and constraints

Maximize source-to-sink delivery subject to link capacities and station processing limits, including the source and sink limits. There are at most `250` stations and `4000` links; parallel links, zero capacities, and internal cycles are legal. All capacities are at most `10^9`.

16.2 Reduction by splitting a station

Replace station `v` by `in(v)` and `out(v)`, with an internal edge

```
in(v) -> out(v), capacity limit[v]
```

Replace each input link `u -> v` of capacity `c` by `out(u) -> in(v)` of capacity `c`. Use `in(1)` as source and `out(n)` as sink. Thus every unit must cross the internal edge of every station it uses, including the endpoints. No artificial infinite capacity is needed.

The code uses `in(v) = 2*v` and `out(v) = 2*v+1` after converting input vertices to 0-based IDs. The transformed graph has `2n` vertices and `n+m` input edges.

```
create every station's internal capacity edge
create every link from its sender's out-node to its receiver's in-node
run Dinic from in(1) to out(n)
```

16.3 Sample walkthrough

The two middle internal edges have capacities `2` and `3`. The links can each carry `9`, but no path can avoid a middle station's internal edge. Send `2` through station 2 and `3` through station 3. Cutting those two internal edges has capacity `5`, certifying that this delivery is maximum.

16.4 Correctness proof

Given a feasible delivery, put every input link's amount on its corresponding transformed link. At an intermediate station, put its total throughput on its internal edge. Conservation balances both split nodes, and the station limit is precisely the internal edge's capacity. The source and sink internal edges carry the total delivered amount and enforce their limits. This constructs a feasible transformed flow of the same value.

Conversely, conservation in the transformed graph forces a station's internal edge to carry its throughput between incoming and outgoing links. Its capacity enforces the processing limit;

transformed link capacities enforce all input link limits. Reading these link flows gives a feasible original delivery of the same value. Since both mappings preserve value, maximum flow solves the problem exactly.

16.5 Dinic reminder and complexity

Store each residual edge with a separate reverse member. A push subtracts from one member and adds to the other. BFS labels positive-residual distances; explicit-stack searches use only edges advancing one level and send a blocking flow. Reset current pointers for every new BFS. When the sink is unreachable, the current flow is maximum by the residual-cut argument from section 45.

With `V=2n` and `E=n+m`, the general bound is `O(V^2*E)` time and `O(V+E)` storage. Actual searches send bottlenecks rather than iterating once per unit of capacity. C++ uses `long long`; Python's explicit stack avoids recursion limits. Parallel links always retain independent reverse pairs.

16.6 Common mistakes and targeted tests

Splitting only middle stations ignores the supplier and receiver limits. Clamping each individual adjacent edge separately does not enforce a shared station limit. Do not merge a cancellation edge with an opposite input link. Fixed cases cover rerouting, a supplier bottleneck, and disconnected vertices. Small random answers come from exhaustive cuts in the split network, not from Dinic. Maximum-size chain/parallel-link cases test a large bottleneck and a closed internal relay, with expected answers `10^9` and `0` respectively.

16.7 Full C++ solution

```cpp
#include <algorithm>
#include <cassert>
#include <queue>
#include <vector>

struct Dinic {
    struct Edge {
        int to;
        long long cap;
    };
    int n;
    std::vector<Edge> edges;
    std::vector<std::vector<int>> adj;
    std::vector<int> level, ptr;

    explicit Dinic(int n) : n(n), adj(n) {}

    int add_edge(int u, int v, long long capacity) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(capacity >= 0);
```

```
        int edge = static_cast<int>(edges.size());
        adj[u].push_back(edge);
        edges.push_back({v, capacity});
        adj[v].push_back(edge ^ 1);
        edges.push_back({u, 0});
        return edge;
    }

    bool bfs(int s, int t) {
        level.assign(n, -1);
        level[s] = 0;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == -1) {
                    level[v] = level[u] + 1;
                    queue.push(v);
                }
            }
        }
        return level[t] != -1;
    }

    long long send(int s, int t, long long limit) {
        std::vector<int> vertices{s}, path;
        std::vector<long long> bottleneck{limit};
        while (!vertices.empty()) {
            int u = vertices.back();
            if (u == t) {
                long long delta = bottleneck.back();
                for (int edge : path) {
                    edges[edge].cap -= delta;
                    edges[edge ^ 1].cap += delta;
                }
                return delta;
            }
            while (ptr[u] < static_cast<int>(adj[u].size())) {
                int edge = adj[u][ptr[u]];
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == level[u] + 1)
                    break;
                ++ptr[u];
            }
            if (ptr[u] == static_cast<int>(adj[u].size())) {
                vertices.pop_back();
                bottleneck.pop_back();
                if (!path.empty()) {
```

```
                    path.pop_back();
                    ++ptr[vertices.back()];
                }
            } else {
                int edge = adj[u][ptr[u]];
                path.push_back(edge);
                vertices.push_back(edges[edge].to);
                bottleneck.push_back(
                    std::min(bottleneck.back(), edges[edge].cap));
            }
        }
        return 0;
    }

    long long max_flow(int s, int t) {
        assert(0 <= s && s < n && 0 <= t && t < n && s != t);
        long long added = 0, limit = 0;
        for (int edge : adj[s]) limit += edges[edge].cap;
        while (bfs(s, t)) {
            ptr.assign(n, 0);
            while (long long delta = send(s, t, limit)) added += delta;
        }
        return added;
    }

    std::vector<bool> reachable(int s) const {
        std::vector<bool> seen(n, false);
        seen[s] = true;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && !seen[v]) {
                    seen[v] = true;
                    queue.push(v);
                }
            }
        }
        return seen;
    }
};

#include <iostream>
int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
```

```cpp
    Dinic network(2 * n);
    for (int v = 0; v < n; ++v) {
        long long limit;
        std::cin >> limit;
        network.add_edge(2 * v, 2 * v + 1, limit);
    }
    for (int i = 0; i < m; ++i) {
        int u, v;
        long long capacity;
        std::cin >> u >> v >> capacity;
        --u;
        --v;
        network.add_edge(2 * u + 1, 2 * v, capacity);
    }
    std::cout << network.max_flow(0, 2 * n - 1) << '\n';
}
```

16.8 Full Python solution

```python
from collections import deque


class Dinic:
    """0-based vertices; integer capacities; paired residual edge IDs."""

    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.to = []
        self.cap = []

    def add_edge(self, u, v, capacity):
        assert 0 <= u < self.n and 0 <= v < self.n
        assert capacity >= 0
        edge = len(self.to)
        self.adj[u].append(edge)
        self.to.append(v)
        self.cap.append(capacity)
        self.adj[v].append(edge ^ 1)
        self.to.append(u)
        self.cap.append(0)
        return edge

    def _bfs(self, s, t):
        self.level = [-1] * self.n
        self.level[s] = 0
        queue = deque([s])
        while queue:
```

```
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and self.level[v] == -1:
                    self.level[v] = self.level[u] + 1
                    queue.append(v)
        return self.level[t] != -1

    def _send(self, s, t, limit):
        vertices = [s]
        path = []
        bottleneck = [limit]
        while vertices:
            u = vertices[-1]
            if u == t:
                delta = bottleneck[-1]
                for edge in path:
                    self.cap[edge] -= delta
                    self.cap[edge ^ 1] += delta
                return delta
            while self.ptr[u] < len(self.adj[u]):
                edge = self.adj[u][self.ptr[u]]
                v = self.to[edge]
                if (self.cap[edge] > 0
                        and self.level[v] == self.level[u] + 1):
                    break
                self.ptr[u] += 1
            if self.ptr[u] == len(self.adj[u]):
                vertices.pop()
                bottleneck.pop()
                if path:
                    path.pop()
                    self.ptr[vertices[-1]] += 1
            else:
                edge = self.adj[u][self.ptr[u]]
                path.append(edge)
                vertices.append(self.to[edge])
                bottleneck.append(min(bottleneck[-1], self.cap[edge]))
        return 0

    def max_flow(self, s, t):
        assert 0 <= s < self.n and 0 <= t < self.n and s != t
        added = 0
        # A finite upper bound on every possible augmentation.
        limit = sum(self.cap[edge] for edge in self.adj[s])
        while self._bfs(s, t):
            self.ptr = [0] * self.n
            while True:
                delta = self._send(s, t, limit)
                if delta == 0:
                    break
```

```
                added += delta
        return added

    def reachable(self, s):
        seen = [False] * self.n
        seen[s] = True
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and not seen[v]:
                    seen[v] = True
                    queue.append(v)
        return seen


import sys


def main():
    data = iter(map(int, sys.stdin.buffer.read().split()))
    n, m = next(data), next(data)
    network = Dinic(2 * n)
    for v in range(n):
        network.add_edge(2 * v, 2 * v + 1, next(data))
    for _ in range(m):
        u, v, capacity = next(data) - 1, next(data) - 1, next(data)
        network.add_edge(2 * u + 1, 2 * v, capacity)
    print(network.max_flow(0, 2 * n - 1))


if __name__ == '__main__':
    main()
```

17 P. Last Alarm

17.1 Restatement and constraints

For every receiver `r`, compute `max_v(a[v] + distance(v,r))` on a weighted tree. Alarms do not compete for edges. With `n <= 200000`, a traversal from every receiver is too slow; edge delays and emission times can each be `10^9`.

17.2 Two directional summaries

Root the tree temporarily at vertex 1. Let `down[u]` be the latest alarm arrival at `u` from its rooted subtree, including `u` itself. Then

```
down[u] = max(a[u], down[child] + edge_delay over all children)
```

Compute this in reverse traversal order. Let `up[u]` be the latest arrival from outside that subtree, or `-1` for an empty outside set. The final answer is `max(down[u], up[u])`.

For child `v` of `u`, outside alarms come from `u` itself, outside `u`'s subtree, or a different child branch. Therefore:

```
up[v] = delay(u,v) + max(a[u], up[u], best other child contribution at u)
```

Keep the two largest child contributions and the child owning the largest. This lets each child exclude its own branch in constant time. Equal largest values from different children must occupy both best slots.

17.3 Sample walkthrough

Root the sample at vertex 1. The downward values are `12,10,0`: station 2's alarm reaches station 1 at time `12`. The outside values for vertices 2 and 3 are `2` and `13`. Taking the larger directional value gives `12,10,13`.

```
build parent-before-child traversal order iteratively
compute down[] in reverse order
for each vertex in forward order:
    answer it from down and up
    find its two best child branches
    pass the best contribution excluding each child down to that child
```

17.4 Correctness proof

Induction from the leaves proves `down[u]`: every subtree alarm is either `u`'s own or lies in exactly one child subtree and must cross its connecting edge. For the forward pass, assume `up[u]` is correct. Removing edge `(u,v)` partitions all alarms outside `v` into exactly the three families in the transition. Their latest arrival at `u`, plus that edge's delay, is exactly the latest outside arrival at `v`. This proves every `up` value by induction. The subtree and outside sets partition all alarms, so their maximum is the required answer at each receiver.

17.5 Complexity and language notes

Every adjacency list is scanned a constant number of times: `O(n)` time and `O(n)` memory. Both implementations use iterative traversals. Times fit signed 64-bit integers: every candidate is at most `10^9 + (n-1)*10^9`. The `-1` sentinel is safe because emission times are nonnegative and delays positive.

17.6 Common mistakes and targeted tests

Using only distance to the farthest station ignores emission times. A single maximum branch cannot safely exclude its owner; preserve a second maximum, including ties. Fixed tests cover a singleton, tied branches, and a late alarm with a large delay. The random oracle independently traverses from every receiver. Full-size tests are a deep path with billion-unit delays and a star whose equal branches exercise exclusion and ties.

17.7 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<ll> alarm(n);
    for (ll& value : alarm) cin >> value;
    vector<vector<pair<int, ll>>> adj(n);
    for (int i = 1; i < n; ++i) {
        int u, v;
        ll delay;
        cin >> u >> v >> delay;
        --u; --v;
        adj[u].push_back({v, delay});
        adj[v].push_back({u, delay});
    }
```

```
    vector<int> parent(n, -1), order{0};
    for (int i = 0; i < n; ++i) {
        int u = order[i];
        for (auto [v, delay] : adj[u]) {
            if (v != parent[u]) {
                parent[v] = u;
                order.push_back(v);
            }
        }
    }
    vector<ll> down = alarm, up(n, -1), answer(n);
    for (int i = n - 1; i >= 0; --i) {
        int u = order[i];
        for (auto [v, delay] : adj[u])
            if (parent[v] == u) down[u] = max(down[u], down[v] + delay);
    }
    for (int u : order) {
        answer[u] = max(down[u], up[u]);
        ll best = -1, second = -1;
        int best_child = -1;
        for (auto [v, delay] : adj[u]) {
            if (parent[v] != u) continue;
            ll candidate = down[v] + delay;
            if (candidate > best) {
                second = best;
                best = candidate;
                best_child = v;
            } else if (candidate > second) {
                second = candidate;
            }
        }
        for (auto [v, delay] : adj[u]) {
            if (parent[v] == u) {
                ll other = v == best_child ? second : best;
                up[v] = delay + max({alarm[u], up[u], other});
            }
        }
    }
    for (int u = 0; u < n; ++u)
        cout << answer[u] << (u + 1 == n ? '\n' : ' ');
}
```

17.8 Full Python solution

```
import sys


def main():
```

```
    data = iter(map(int, sys.stdin.buffer.read().split()))
    n = next(data)
    alarm = [next(data) for _ in range(n)]
    adj = [[] for _ in range(n)]
    for _ in range(n - 1):
        u, v, delay = next(data) - 1, next(data) - 1, next(data)
        adj[u].append((v, delay))
        adj[v].append((u, delay))

    parent = [-1] * n
    order = [0]
    for u in order:
        for v, _ in adj[u]:
            if v != parent[u]:
                parent[v] = u
                order.append(v)

    down = alarm.copy()
    for u in reversed(order):
        for v, delay in adj[u]:
            if parent[v] == u:
                down[u] = max(down[u], down[v] + delay)

    up = [-1] * n
    answer = [0] * n
    for u in order:
        answer[u] = max(down[u], up[u])
        best = second = -1
        best_child = -1
        for v, delay in adj[u]:
            if parent[v] != u:
                continue
            candidate = down[v] + delay
            if candidate > best:
                second, best = best, candidate
                best_child = v
            elif candidate > second:
                second = candidate
        for v, delay in adj[u]:
            if parent[v] == u:
                other = second if v == best_child else best
                up[v] = delay + max(alarm[u], up[u], other)
    print(' '.join(map(str, answer)))


if __name__ == '__main__':
    main()
```

18 Q. Exact Beacon Codes

18.1 Restatement and constraints

Count index subsets having both cardinality `k` and XOR `target`. There are up to `40` beacons, so `2^n` enumeration is too large, while a code-indexed DP would need up to `2^30` states. Equal values remain distinct beacons.

18.2 Split by index, retain both summaries

Divide the indices into two halves of at most 20 elements. Enumerate left subsets and store their frequencies by `(selected count, XOR)`. For a right subset with count `c` and XOR `x`, its complement requirements are

```
left count = k - c
left XOR   = target XOR x
```

The second identity follows because XOR is its own inverse. Add the stored frequency, rather than just checking whether a matching key exists.

```
enumerate all left subsets into frequency[count][xor]
answer = 0
for every right subset:
    if the required left count is legal:
        answer += frequency[required count][required xor]
```

Recursive include/skip enumeration has depth at most 20. Branches already using more than `k` elements may be discarded. The empty subset is enumerated on both sides with count zero and XOR zero.

18.3 Sample walkthrough

The left half has codes `1,1`, and the right half `2,2`. For target zero and count two, either take both left beacons and no right beacon, or both right beacons and no left beacon. The cross pairs have XOR `3` and do not qualify. The answer is `2`.

18.4 Correctness proof

Every full index subset decomposes uniquely into its intersections with the two disjoint halves. Its counts add and its codes combine by XOR. Thus a right subset pairs with exactly those left subsets in its required frequency bucket. The algorithm counts each of these pairs once, counts no invalid pair, and visits the unique right component of every valid full subset. The total is therefore exact, including multiplicities from equal codes.

18.5 Complexity and language notes

With expected constant-time dictionary operations, time is `O(2^floor(n/2) + 2^ceil(n/2))`, and memory is `O(2^floor(n/2))`. C++ uses `unordered_map<int,long long>` buckets and a 64-bit total; Python uses dictionaries and integer counts. There is no modulus. The answer is at most `C(40,20) = 137846528820`, which fits signed 64-bit arithmetic.

18.6 Common mistakes and targeted tests

Do not collapse identical beacons, discard subset frequencies, omit cardinality, or forget `k=0`. Fixed tests include both empty-subset outcomes, multiplicities, and the highest allowed code bits. The random oracle enumerates full masks, filters by cardinality, and directly XORs selected codes. Full-size tests use 40 zero codes and two copies of each of 20 independent bits. Their answers are independently `C(40,20)` and `C(20,10)`; the latter requires selecting both copies of exactly ten bits.

18.7 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k, target;
    cin >> n >> k >> target;
    vector<int> values(n);
    for (int& value : values) cin >> value;
    int middle = n / 2;
    vector<unordered_map<int, long long>> frequency(middle + 1);
    function<void(int, int, int)> collect = [&](int i, int used, int code) {
        if (used > k) return;
        if (i == middle) {
            ++frequency[used][code];
            return;
        }
        collect(i + 1, used, code);
        collect(i + 1, used + 1, code ^ values[i]);
```

```
    };
    function<long long(int, int, int)> combine = [&](
        int i, int used, int code) {
        if (used > k) return 0LL;
        if (i == n) {
            int need = k - used;
            if (need < 0 || need > middle) return 0LL;
            auto it = frequency[need].find(target ^ code);
            return it == frequency[need].end() ? 0LL : it->second;
        }
        return combine(i + 1, used, code)
             + combine(i + 1, used + 1, code ^ values[i]);
    };
    collect(0, 0, 0);
    cout << combine(middle, 0, 0) << '\n';
}
```

18.8 Full Python solution

```
import sys


def count_subsets(values, wanted_count, target):
    middle = len(values) // 2
    left, right = values[:middle], values[middle:]
    frequency = [{} for _ in range(len(left) + 1)]

    def collect(i, used, code):
        if used > wanted_count:
            return
        if i == len(left):
            group = frequency[used]
            group[code] = group.get(code, 0) + 1
            return
        collect(i + 1, used, code)
        collect(i + 1, used + 1, code ^ left[i])

    def combine(i, used, code):
        if used > wanted_count:
            return 0
        if i == len(right):
            need = wanted_count - used
            if 0 <= need < len(frequency):
                return frequency[need].get(target ^ code, 0)
            return 0
        return (combine(i + 1, used, code)
                + combine(i + 1, used + 1, code ^ right[i]))
```

```
    collect(0, 0, 0)
    return combine(0, 0, 0)


def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, k, target = data[:3]
    print(count_subsets(data[3:3 + n], k, target))


if __name__ == '__main__':
    main()
```

19 R. Warehouse Blocks

19.1 Restatement and constraints

Partition sorted customers into exactly `k` nonempty contiguous blocks, each served by one freely placed warehouse. With `n <= 30000` and `k <= 30`, the straightforward `O(kn^2)` partition DP is too slow.

19.2 Optimal cost of one block

A warehouse minimizing the sum of absolute distances can be placed at a median customer. Pair the leftmost and rightmost customer, then the next pair: for each pair, every coordinate between them minimizes their combined distance. A median lies in all these intervals, so it minimizes the total. For an odd block, the unpaired middle customer also favors that median.

Using 0-based half-open block `[l,r)`, let `m=(l+r-1)//2`, the lower median. With prefix sums `P`, its cost is

```
C(l,r) = x[m]*(m-l) - (P[m]-P[l])
       + (P[r]-P[m+1]) - x[m]*(r-m-1)
```

This costs `O(1)` to evaluate after `O(n)` preprocessing.

19.3 Recurrence and why its optima are ordered

Let `dp[g][i]` be the minimum cost of serving the first `i` customers with exactly `g` blocks. Initialize only `dp[0][0]=0`; other states are infinity. The last block begins after a prefix of `j` customers:

```
dp[g][i] = min(dp[g-1][j] + C(j,i)) for g-1 <= j < i
```

We must prove the cost has the Monge property before restricting split ranges. For this paragraph only, use inclusive 1-based interval cost `D(l,r)`. Appending a customer has marginal cost

```
D(l,r+1) - D(l,r) = x[r+1] - x[floor((l+r+1)/2)]
```

This follows directly by separating even and odd block sizes in the median formula. As `l` increases, the median index moves right, so this marginal cost cannot increase. Summing such marginal costs gives, for `a <= b <= c <= d`,

```
D(a,c) + D(b,d) <= D(a,d) + D(b,c)
```

This is exactly the Monge inequality for the transition-cost matrix. Adding `dp[g-1][j]` to an entire candidate row preserves that inequality. Consequently, choosing the smallest minimizing split gives `opt[g][i] <= opt[g][i+1]`. For completeness, if two smallest minima crossed, apply the inequality to the two candidates and outputs. Together with their minimizing inequalities, it forces both candidates to tie at both outputs, contradicting the choice of the smallest split at the earlier output.

19.4 Divide-and-conquer algorithm and sample

For one layer, evaluate the middle output in its allowed candidate range. If its best split is `best`, recurse on left outputs with upper candidate bound `best`, and right outputs with lower candidate bound `best`. Search candidates in increasing order and replace the best only on a strict improvement to preserve the smallest-minimizer tie rule.

```
previous[0] = 0; all other states = infinity
for g = 1..k:
    compute outputs g..n by divide and conquer
    search only j >= g-1 and j < output
    keep two DP rows
return dp[k][n]
```

In the sample, the first three customers use median `2` at cost `3`, and the last two use median `10` at cost `1`. Split `j=3` gives total `4`.

19.5 Correctness proof

The recurrence covers every possible last block and combines it with an optimal prefix, so it equals the true optimum. At each divide-and-conquer call, assume the candidate interval contains a smallest optimum for all its outputs. The middle search finds its optimum exactly. Monotonicity then proves the restricted left and right candidate intervals still contain their optima. Induction over recursive calls preserves the recurrence's answers, and over layers yields the required optimum for exactly `k` nonempty blocks.

19.6 Complexity and language notes

Each layer uses `O(n log n)` candidate evaluations, for `O(kn log n)` time and `O(n)` storage. Recursive computation depth is only `O(log n)`, safe in Python; it is not recursion along all customers. C++ uses `long long` for prefixes, products, and costs. Both references use half-open block costs and start layer `g` at output `g`, excluding empty blocks.

19.7 Common mistakes and targeted tests

An average minimizes squared distances, not these absolute distances. Do not apply monotone optimization before proving the actual median cost's inequality. Fixed cases cover one customer, one warehouse per customer, even-sized blocks, and duplicate positions. The small oracle enumerates every cut combination and tries every customer coordinate as warehouse, sharing neither optimized DP nor its cost formula. Maximum-size tests have equally spaced customers and all coordinates equal. For equally spaced blocks of size `h`, cost is `gap*floor(h*h/4)`; convexity makes balanced block sizes optimal. The generated `n=30000,k=30` instance therefore has independently known answer `225000000000`.

19.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<ll> positions(n), prefix(n + 1);
    for (int i = 0; i < n; ++i) {
        cin >> positions[i];
        prefix[i + 1] = prefix[i] + positions[i];
    }
    auto cost = [&](int left, int right) {
        int median = (left + right - 1) / 2;
        ll x = positions[median];
        return x * (median - left) - (prefix[median] - prefix[left])
             + prefix[right] - prefix[median + 1] - x * (right - median - 1);
    };
    const ll INF = 1LL << 62;
    vector<ll> previous(n + 1, INF);
    previous[0] = 0;
    for (int group = 1; group <= k; ++group) {
        vector<ll> current(n + 1, INF);
        function<void(int, int, int, int)> compute =
            [&](int low, int high, int opt_low, int opt_high) {
                if (low > high) return;
                int middle = (low + high) / 2;
                ll best_value = INF;
                int best_split = opt_low;
                int last_split = min(opt_high, middle - 1);
                for (int split = opt_low; split <= last_split; ++split) {
                    if (previous[split] == INF) continue;
                    ll candidate = previous[split] + cost(split, middle);
                    if (candidate < best_value) {
```

```
                        best_value = candidate;
                        best_split = split;
                    }
                }
                current[middle] = best_value;
                compute(low, middle - 1, opt_low, best_split);
                compute(middle + 1, high, best_split, opt_high);
            };
        compute(group, n, group - 1, n - 1);
        previous.swap(current);
    }
    cout << previous[n] << '\n';
}
```

19.9 Full Python solution

```
import sys


def minimum_cost(positions, groups):
    n = len(positions)
    prefix = [0]
    for x in positions:
        prefix.append(prefix[-1] + x)

    def cost(left, right):
        # Half-open block [left, right), using its lower median.
        median = (left + right - 1) // 2
        x = positions[median]
        return (x * (median - left) - (prefix[median] - prefix[left])
                + prefix[right] - prefix[median + 1]
                - x * (right - median - 1))

    infinity = 10**30
    previous = [0] + [infinity] * n
    for group in range(1, groups + 1):
        current = [infinity] * (n + 1)

        def compute(low, high, opt_low, opt_high):
            if low > high:
                return
            middle = (low + high) // 2
            best_value = infinity
            best_split = opt_low
            for split in range(opt_low, min(opt_high, middle - 1) + 1):
                candidate = previous[split] + cost(split, middle)
                if candidate < best_value:
                    best_value = candidate
                    best_split = split
```

```
            current[middle] = best_value
            compute(low, middle - 1, opt_low, best_split)
            compute(middle + 1, high, best_split, opt_high)

        compute(group, n, group - 1, n - 1)
        previous = current
    return previous[n]


def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, k = data[:2]
    print(minimum_cost(data[2:2 + n], k))


if __name__ == '__main__':
    main()
```

20 S. Parabolic Signals

20.1 Restatement and constraints

At each receiver coordinate, maximize `b[i]-(x-p[i])^2` over all transmitters. There can be `200000` transmitters and queries, with arbitrary repeated signed coordinates. Directly comparing every transmitter for every receiver is quadratic. The answer may be negative.

20.2 Turn parabolas into a line envelope

All parabolas have the same quadratic coefficient. Expanding gives

```
b[i] - (x-p[i])^2 = -x^2 + 2*p[i]*x + b[i] - p[i]^2
```

The term `-x^2` is independent of the transmitter. To reuse a minimum line container, negate the remaining part:

```
line slope     = -2*p[i]
line intercept = p[i]^2 - b[i]
answer(x)      = -min_line_value(x) - x^2
```

All transmitters are available from the start, so insert every line before answering any query. No DP order restriction is involved.

20.3 Discrete Li Chao operations

Sort the distinct query coordinates. A tree node owns an interval of their indices and stores at most one line. At insertion, keep the better line at the middle coordinate in the node. The displaced line can improve on the retained line on at most one side, because the difference of two lines is linear. Compare at the two endpoints, recurse into the improving half, or discard the line if neither endpoint improves. Empty nodes accept immediately.

A query walks the root-to-leaf path containing its coordinate and takes the minimum among stored lines on that path. Index compression does not replace coordinates by ranks in line evaluation: always evaluate at the actual `x`.

```
collect and sort unique receiver coordinates
insert (-2*p, p*p-b) for every transmitter
for each receiver in its original order:
    print -minimum_line_value(x) - x*x
```

20.4 Sample walkthrough

Transmitters `(0,5)` and `(4,8)` become lines `-5` and `-8*x+8`.

Receiver x	Minimum line value	Subtract x squared after negation	Answer
0	-5	5 - 0	5
2	-8	8 - 4	4
5	-32	32 - 25	7

20.5 Correctness proof

The expansion is an exact identity for each transmitter. Thus maximizing its strength is equivalent to negating the minimum transformed line value and subtracting `x^2`.

Insertion preserves the minimum envelope at every query coordinate: the retained line dominates the displaced one at the midpoint; outside a possible single improving side the displaced line cannot matter, and on that side it is retained recursively. Previously stored alternatives remain available in their subtrees. Induction over insertions proves that each coordinate's true minimum is represented somewhere along its root-to-leaf path. Querying that path therefore returns the exact minimum, and the identity yields the exact maximum signal strength.

20.6 Complexity and language notes

Let `U <= q` be the number of distinct receiver coordinates. Sorting takes `O(q log q)` time; insertions and queries take `O((n+q)*log(U+1))`, with `O(n+q)` storage including input and output. The tree itself uses `O(U)` slots.

C++ evaluates all products in `long long`. Under the bounds, a squared distance is at most `4*10^12` and a strength has absolute value at most `5*10^12`. Python uses integer arithmetic. Explicit empty slots avoid evaluating a fake infinite line. Equal slopes and a single distinct coordinate need no special intersection division.

20.7 Common mistakes and targeted tests

Do not clamp negative strengths to zero, evaluate at compressed ranks, assume sorted input, or insert transmitters only after answering some receivers. Fixed cases cover negative-only answers, equal slopes, a single query coordinate, ties, and large signed products. The random oracle evaluates the original parabolic formula for every pair. Full-size tests have many different winning transmitters and many equal slopes with large positive and negative answers.

20.8 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct Line {
    ll slope = 0, intercept = 0;
    ll value(ll x) const { return slope * x + intercept; }
};

struct LiChao {
    vector<ll> x;
    vector<Line> tree;
    vector<bool> occupied;
    explicit LiChao(vector<ll> coordinates) : x(move(coordinates)) {
        sort(x.begin(), x.end());
        x.erase(unique(x.begin(), x.end()), x.end());
        tree.resize(4 * x.size());
        occupied.assign(4 * x.size(), false);
    }
    void add(Line line, int node, int left, int right) {
        if (!occupied[node]) {
            occupied[node] = true;
            tree[node] = line;
            return;
        }
        int middle = (left + right) / 2;
        if (line.value(x[middle]) < tree[node].value(x[middle]))
            swap(line, tree[node]);
        if (left == right) return;
        if (line.value(x[left]) < tree[node].value(x[left]))
            add(line, 2 * node, left, middle);
        else if (line.value(x[right]) < tree[node].value(x[right]))
            add(line, 2 * node + 1, middle + 1, right);
    }
    void add(Line line) { add(line, 1, 0, int(x.size()) - 1); }
    ll query(ll at) const {
        ll answer = 1LL << 62;
        int node = 1, left = 0, right = int(x.size()) - 1;
        while (true) {
            if (occupied[node]) answer = min(answer, tree[node].value(at));
            if (left == right) return answer;
            int middle = (left + right) / 2;
            if (at <= x[middle]) { node *= 2; right = middle; }
            else { node = 2 * node + 1; left = middle + 1; }
        }
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
    int n, q;
    cin >> n >> q;
    vector<pair<ll, ll>> stations(n);
    for (auto& [position, power] : stations) cin >> position >> power;
    vector<ll> queries(q);
    for (ll& x : queries) cin >> x;
    LiChao hull(queries);
    for (auto [position, power] : stations)
        hull.add({-2 * position, position * position - power});
    for (ll x : queries) cout << -hull.query(x) - x * x << '\n';
}
```

20.9 Full Python solution

```
import sys


class LiChao:
    def __init__(self, coordinates):
        self.x = sorted(set(coordinates))
        self.tree = [None] * (4 * len(self.x))

    @staticmethod
    def value(line, x):
        return line[0] * x + line[1]

    def add(self, line, node=1, left=0, right=None):
        if right is None:
            right = len(self.x) - 1
        if self.tree[node] is None:
            self.tree[node] = line
            return
        middle = (left + right) // 2
        if (self.value(line, self.x[middle])
                < self.value(self.tree[node], self.x[middle])):
            self.tree[node], line = line, self.tree[node]
        if left == right:
            return
        if (self.value(line, self.x[left])
                < self.value(self.tree[node], self.x[left])):
            self.add(line, 2 * node, left, middle)
        elif (self.value(line, self.x[right])
                < self.value(self.tree[node], self.x[right])):
            self.add(line, 2 * node + 1, middle + 1, right)

    def query(self, x):
        answer = 10**30
        node, left, right = 1, 0, len(self.x) - 1
        while True:
```

```
            if self.tree[node] is not None:
                answer = min(answer, self.value(self.tree[node], x))
            if left == right:
                return answer
            middle = (left + right) // 2
            if x <= self.x[middle]:
                node, right = 2 * node, middle
            else:
                node, left = 2 * node + 1, middle + 1


def main():
    data = iter(map(int, sys.stdin.buffer.read().split()))
    n, q = next(data), next(data)
    stations = [(next(data), next(data)) for _ in range(n)]
    queries = [next(data) for _ in range(q)]
    hull = LiChao(queries)
    for position, power in stations:
        hull.add((-2 * position, position * position - power))
    print('\n'.join(str(-hull.query(x) - x * x) for x in queries))


if __name__ == '__main__':
    main()
```

21 Testing discussion

The seven new packages use independent small oracles: explicit rotations, exhaustive substring partitions, enumerated cuts, traversals from every tree receiver, full subset masks, all warehouse cut combinations and positions, and direct parabolic evaluations. None calls a reference solution to obtain its expected outputs. Each also supplies fixed edge cases and two full-limit performance workloads with construction-based answers.

Run all nineteen references in both languages with the standard section checker. Performance cases are included even when randomized testing is disabled through the underlying judge's `--random-count 0` option. A fast implementation should pass both oracle and performance cases; neither class of test substitutes for the other.

For the older packages, also compare tiny games with direct mex evaluation, CRT with brute-force residue search, Fibonacci with iterative recurrence, and optimization tasks with their direct formulations. Keep impossible cases, empty selections, repeated values, negative quantities, and large products as separate regression targets.

22 What to carry forward

Identify the exact objects being counted or optimized before choosing the structure. Preserve multiplicities when counting, distinguish state IDs from string lengths, and prove reductions in both directions. Greedy choices and restricted DP searches need their own proofs. Finally, choose tests that expose the proposed algorithm's failure modes at both small and maximum input sizes.